ONLINE SHOPPING APPLICATION DATABASE

A PROJECT REPORT

Submitted by

MITHRA S

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
SAMAYAPURAM, TRICHY

ANNA UNIVERSITY
CHENNAI 600 025

MAY 2026

ONLINE SHOPPING APPLICATION DATABASE

PROJECT FINAL DOCUMENT

Submitted by

MITHRA S (8115U24AM036)

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Under the Guidance of

Mr. P. NAGARAJ M.E.,

Department of Artificial Intelligence and Machine Learning

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this project report titled **"ONLINE SHOPPING APPLICATION DATABASE"** is the bonafide work of **MITHRA S (8115U24AM036)** who carried out the work under my supervision.

SIGNATURE

Dr. B. KIRAN BALA M.E.,M.B.A.,Ph.D.,

HEAD OF THE DEPARTMENT
ASSOCIATE PROFESSOR,

Department of Artificial Intelligence and Machine Learning,
K. Ramakrishnan College of Engineering, (Autonomous)
Samayapuram, Trichy.

SIGNATURE

Mr. P. NAGARAJ M.E.,

SUPERVISOR
ASSISTANT PROFESSOR,

Department of Artificial Intelligence and Machine Learning,
K. Ramakrishnan College of Engineering, (Autonomous)
Samayapuram, Trichy.

SIGNATURE OF INTERNAL EXAMINER

NAME:
DATE:

SIGNATURE OF EXTERNAL EXAMINER

NAME:
DATE:

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI

DECLARATION BY THE CANDIDATE

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the project Viva-Voice held at K. Ramakrishnan College of Engineering on _____

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I thank the almighty GOD, without whom it would not have been possible for me to complete my project.

I wish to address my profound gratitude to **Dr.K.RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering (Autonomous), who encouraged and gave me all help throughout the course.

I extend my hearty gratitude and thanks to my honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.,** K. Ramakrishnan College of Engineering (Autonomous).

I am glad to thank my Principal **Dr.D.SRINIVASAN, M.E., Ph.D.,FIE., MIIW., MISTE., MISAE., C.Engg,** for giving me permission to carry out this project.

I wish to convey my sincere thanks to **Dr.B.KIRAN BALA, M.E., M.B.A., Ph.D.,** Head of the Department, Artificial Intelligence and Machine Learning, for giving me constant encouragement and advice throughout the course.

I am grateful to **Mr.P.NAGARAJ, M.E., Assistant Professor**, Artificial Intelligence and Machine Learning, K. Ramakrishnan College of Engineering (Autonomous), for his guidance and valuable suggestions during the course of study.

Finally, I sincerely acknowledged in no less terms all my staff members, my parents and, friends for their co-operation and help at various stages of this project work.

MITHRA S (8115U24AM036)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

VISION

To become a renowned hub for AIML technologies to producing highly talented globally recognizable technocrats to meet industrial needs and societal expectation.

MISSION

Mission of the Department

M1 To impart advanced education in AI and Machine Learning, built upon a foundation in Computer Science and Engineering.

M2 To foster Experiential learning equips students with engineering skills to tackle real-world problems.

M3 To promote collaborative innovation in AI, machine learning, and related research and development with industries.

M4 To provide an enjoyable environment for pursuing excellence while upholding strong personal and professional values and ethics.

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

PEO1 Excel in technical abilities to build intelligent systems in the fields of AI & ML in order to find new opportunities.

PEO2 Embrace new technology to solve real-world problems, whether alone or as a team, while prioritizing ethics and societal benefits.

PEO3 Accept lifelong learning to expand future opportunities in research and product development.

PROGRAM SPECIFIC OUTCOMES (PSO's)

PSO1 Expertise in tailoring ML algorithms and models to excel in designated applications and fields.

PSO2 Ability to conduct research, contributing to machine learning advancements and innovations that tackle emerging societal challenges.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO8 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO9 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO10 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO11 Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

The **Online Shopping Application Database System** is a Python-based application developed using Stream lit and SQLite to provide a secure and efficient platform for managing online transactions. The system allows users to register, log in, and perform payment operations while maintaining all data in a structured database. It ensures proper data management by storing user details and transaction records in an organized manner. The application includes important features such as secure user authentication, OTP-based transaction verification, and transaction history management. It also incorporates basic fraud detection mechanisms by analyzing factors like high transaction amounts, device changes, and location variations to prevent suspicious activities. The system uses SQL queries for efficient data storage and retrieval, ensuring quick response and reliable performance. The user interface is simple and interactive, allowing users to easily navigate through different modules such as login, payment, and transaction history. Overall, the system demonstrates the practical implementation of database concepts along with security features in an online shopping environment. It provides a strong foundation for developing advanced e-commerce applications with enhanced scalability and functionality.

ABSTRACT WITH POs AND PSOs MAPPING

ABSTRACT	POs MAPPED	PSOs MAPPED
The **Online Shopping Application Database System** is a Python-based application that helps manage online shopping operations such as user registration, product management, order processing, payment transactions, and transaction history efficiently. It uses Python for application logic and SQLite (SQL) for backend data storage, where all data is maintained in a structured and normalized database to ensure accuracy and reduce redundancy. The system supports secure user authentication, shopping cart management, and CRUD operations (Create, Read, Update, Delete) for handling users, products, and transactions. It also includes secure payment features such as OTP verification and basic fraud detection mechanisms like monitoring transaction amount, device, and location to enhance security and reliability. Additionally, the system provides a transaction history module to track all user activities, making the application user-friendly, scalable, and suitable for academic projects and small to medium-scale e-commerce applications.	**PO1-3** **PO2-3** **PO3-3** **PO4-3** **PO5-3** **PO6-3** **PO7-3** **PO8-3** **PO9-3** **PO10-3** **PO11-3**	**PSO1 – 3** **PSO2 – 3**

Note: 1- Low, 2-Medium, 3- High

TABLE OF CONTENTS

CHAPTER No.		TITLE	PAGE No.
		ABSTRACT	**viii**
		LIST OF ABBREVIATIONS	**xii**
		LIST OF FIGURES	**xiii**
1		**INTRODUCTION**	**1**
	1.1	Objective	1
	1.2	Scope of the Project	2
	1.3	Overview of the System	3
	1.4	Importance of DBMS in shopping systems	4
	1.5	Technologies Used	6
2		**LITERATURE SURVEY**	**8**
	2.1	Requirement Analysis	8
	2.2	Entity-Relationship (ER) Diagram	9
	2.3	Relational Schema	10
	2.4	Normalization Process	11
	2.5	System Architecture Overview	12
3		**MODULE DESCRIPTION**	**14**
	3.1	User Registration Module	14
	3.2	User Login Module	15
	3.3	Payment Module	15
	3.4	OTP Verification Module	16
	3.5	Transaction Management Module	16
	3.6	Transaction History Module	17

	3.7	Fraud Detection Module	18
4		**DATABASE IMPLEMENTATION**	**19**
	4.1	Table Creation Scripts	19
	4.2	Constraints and Relationships	21
	4.3	Backup & Recovery Considerations	22
5		**RESULTS AND DISCUSSION**	**24**
	5.1	Performance Analysis	24
	5.2	Observations and Interpretations	25
6		**CONCLUSION AND FUTURE WORK**	**28**
	6.1	Summary of Outcomes	28
	6.2	Future Scope and Enhancements	29
7		**APPENDIX**	**31**
		APPENDIX A – Source Code	31
		APPENDIX B - Screenshots	46
		REFERENCES	**49**

LIST OF ABBREVIATIONS

S. NO.	ACRONYM	ABBREVIATION
1	CRUD	Create, Read, Update, Delete
2	DBMS	Database Management System
3	RDBMS	Relational Database Management System
4	SQL	Structured Query Language
5	ER	Entity Relationship
6	CSV	Comma Separated Values
7	IDE	Integrated Development Environment
8	API	Application Programming Interface
9	PK	Primary Key
10	FK	Foreign Key

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
2.2.1	Entity Relationship	9
2.3.1	Relational Schema	10
2.5.1	System Architecture	13
B.1	Login Page	46
B.2	Dashboard Overview	46
B.3	Product Listing Page	47
B.4	Cart Page	47
B.5	Cart After Purchase	48
B.6	Purchase History Page	48

CHAPTER 1
INTRODUCTION

1.1 OBJECTIVE

The primary objective of the **Online Shopping Application Database System** is to design and develop a secure, efficient, and user-friendly application that manages various online shopping operations such as user registration, product browsing, order processing, and payment transactions. In the current digital era, e-commerce platforms play a vital role in connecting customers and businesses, making it essential to maintain accurate and well-organized data for smooth operations.

This system aims to provide a structured and centralized platform for storing and managing large volumes of data related to users, products, and transactions. By implementing a database-driven approach, the system reduces manual work, minimizes human errors, and improves the speed and accuracy of operations. The application ensures that users can easily navigate through the system and perform their tasks without any complexity.

The system is developed with the objective of implementing all essential CRUD (Create, Read, Update, Delete) functionalities. Users can register, log in securely, view products, add items to the cart, and complete transactions. The system also maintains a complete record of transactions, allowing users to view their purchase history at any time. This enhances transparency and improves user experience.

Additionally, the system incorporates security mechanisms such as OTP-based verification to ensure safe and secure transactions. Fraud detection features like monitoring transaction amount, device, and location are included to identify unusual activities and prevent unauthorized access.

Furthermore, the project aims to demonstrate the practical implementation of Database Management System (DBMS) concepts such as normalization, relational schema design, and data integrity. The overall objective is to develop a scalable and efficient online shopping system that can be extended for real-world applications and future enhancements.

1.2 SCOPE OF THE PROJECT

The **Online Shopping Application Database System** is designed to provide a complete and efficient platform for managing online shopping activities. The scope of this project includes various functionalities such as user management, product handling, shopping cart operations, secure payment processing, and transaction history maintenance. It is suitable for academic purposes as well as small to medium-scale e-commerce applications.

The system allows users to register and log in securely, ensuring that only authorized users can access the application. Once logged in, users can browse available products, view details, and add items to their shopping cart. The system provides an interactive and user-friendly interface, making it easy for users to navigate and perform operations efficiently.

The project includes a secure payment module that ensures safe transaction processing using OTP verification. This feature adds an extra layer of security by confirming user identity before completing the transaction. In addition, the system implements basic fraud detection mechanisms by analyzing transaction patterns such as high transaction amounts, unusual device usage, and changes in location. These features help in preventing fraudulent activities and maintaining system integrity.

The system also supports CRUD operations for managing users, products, and transactions. All data is stored in a structured database, enabling efficient storage, retrieval, and updating of information. The database design ensures minimal redundancy and maximum consistency by applying normalization techniques.

From a technical perspective, the system is implemented using Python (Streamlit) for the front-end interface and SQLite as the backend database. The integration of these technologies provides a lightweight, efficient, and easy-to-use application.

The scope of the project can be extended further by adding advanced features such as online payment gateway integration, cloud-based storage, product recommendation systems, and mobile application support. It can also be enhanced with real-time analytics and reporting tools for better decision-making.

Overall, the project provides a strong foundation for developing a full-fledged online shopping platform while demonstrating the effective use of DBMS concepts in real-world applications.

1.3 OVERVIEW OF THE SYSTEM

The Online Shopping Application Database System is a database-driven application designed to facilitate online shopping operations in a secure and efficient manner. The system provides an easy-to-use interface where users can perform activities such as registration, login, product browsing, and payment processing.

The system follows a structured architecture in which the user interacts with the front-end interface, which communicates with the backend logic and database. The front-end is developed using Python (Streamlit), which provides an interactive user interface. The backend logic handles user requests, processes data, and interacts with the database. SQLite is used as the database to store all application-related data in a structured format.

The system supports different functionalities that work together to provide a complete online shopping experience:

- User Authentication: Users can register and log in securely using their credentials.
- Product Management: Displays product details such as name, price, and availability.
- Shopping Cart: Allows users to select and manage items before making a purchase.
- Payment System: Handles transactions securely using OTP verification.

- Transaction History: Stores all transaction details for future reference.

The system also includes security features such as fraud detection by monitoring transaction amount, device, and location. These features help in identifying suspicious activities and preventing unauthorized transactions.

The backend database plays a crucial role in storing and managing data efficiently. All information related to users, products, and transactions is stored in tables with proper relationships and constraints. This ensures data integrity, consistency, and efficient retrieval of information.

The application is designed to be simple, reliable, and scalable. It provides a smooth user experience while ensuring secure data handling. The modular structure of the system makes it easy to maintain and upgrade with additional features in the future.

1.4 IMPORTANCE OF DBMS IN ONLINE SHOPPING SYSTEMS

A **Database Management System (DBMS)** plays a crucial role in the development and functioning of an Online Shopping Application. In an e-commerce environment, a large amount of data is continuously generated, including user information, product details, orders, payments, and transaction history. Managing this data manually is difficult and error-prone. A DBMS provides an efficient way to store, organize, and manage this data in a structured format.

One of the major roles of DBMS in an online shopping system is **data organization**. All data such as users, products, orders, and transactions are stored in separate tables with proper relationships. This structured storage helps in easy retrieval and management of data. For example, user details are stored in one table, product details in another, and transactions in another, making the system more organized.

Another important aspect is **data integrity and accuracy**. DBMS ensures that the data stored in the database is accurate and consistent by applying constraints such as primary

keys, foreign keys, and validation rules. These constraints prevent duplicate entries, invalid data, and inconsistencies, ensuring reliable system performance.

DBMS also provides **efficient data retrieval** using SQL queries. In an online shopping system, users frequently search for products, view order details, and check transaction history. With the help of optimized queries, data can be retrieved quickly, improving system performance and user experience.

Security is another critical factor where DBMS plays an important role. Sensitive information such as user credentials and transaction details are stored securely in the database. Access control mechanisms ensure that only authorized users can access or modify the data. In addition, features like encryption and authentication help protect data from unauthorized access.

The DBMS also supports **transaction management**, which is essential for handling online payments. It ensures that transactions are completed successfully and maintains consistency even in case of system failures. If any error occurs during a transaction, the DBMS can roll back the changes to maintain data integrity.

Data redundancy reduction is achieved through normalization techniques. By organizing data into multiple related tables, duplication of data is minimized, leading to efficient storage and better performance. This also helps in maintaining consistency across the system.

Another advantage of DBMS is **scalability**. As the number of users and transactions increases, the system can handle large volumes of data without affecting performance. This makes it suitable for growing e-commerce platforms.

DBMS also supports **backup and recovery mechanisms**, which are essential for protecting data from loss due to system failures, crashes, or human errors. Regular backups ensure that data can be restored when needed.

In addition, DBMS enables **data analysis and reporting**, which helps administrators understand user behavior, sales trends, and system performance. This information can be used to improve business strategies and enhance user experience.

Overall, the DBMS acts as the backbone of the Online Shopping Application, ensuring efficient data management, security, reliability, and scalability. It plays a vital role in maintaining the smooth functioning of the entire system.

1.5 TECHNOLOGIES USED

The **Online Shopping Application Database System** is developed using a combination of modern technologies that work together to provide an efficient, secure, and user-friendly application. These technologies are categorized into front-end, back-end, database, and supporting tools.

Front-End Technology (User Interface)

Python (Streamlit):

Streamlit is used to design the user interface of the application. It provides an interactive and easy-to-use environment for building web-based applications. Using Streamlit, various components such as text inputs, buttons, forms, and menus are created to allow users to interact with the system. The interface is designed to be simple and user-friendly so that users can perform operations like login, product selection, and payment easily.

The front-end plays an important role in enhancing user experience by providing smooth navigation and clear display of information. It ensures that users can access all features without technical complexity.

Back-End Technology (Application Logic)

Python:

Python is used to implement the core logic of the application. It handles all functionalities such as user authentication, product management, payment processing, and transaction handling. Python is chosen because of its simplicity, readability, and wide support for libraries.

The backend processes user inputs, performs validations, and communicates with the

database to store and retrieve data. It ensures that all operations are executed correctly and efficiently.

Authentication and Security:

The system includes user authentication mechanisms to verify login credentials. OTP (One-Time Password) verification is used during transactions to enhance security. Basic fraud detection techniques are also implemented to monitor suspicious activities such as unusual transaction amounts, device changes, and location differences.

Database Technology (Data Storage)

SQLite (SQL):

SQLite is used as the backend database for storing all application data. It is a lightweight, serverless database that is easy to integrate with Python. All data such as users, products, and transactions are stored in structured tables.

SQL (Structured Query Language):

SQL is used to perform database operations such as INSERT, SELECT, UPDATE, and DELETE. These operations help in managing data efficiently. SQL also supports advanced queries for retrieving specific information, such as transaction history or product details.

The database is designed using relational schema and normalization techniques to ensure data consistency and reduce redundancy.

In addition to the core technologies, the system is designed using a **modular development approach**, where each functionality such as login, product management, payment processing, and transaction tracking is implemented as a separate module. This approach improves code maintainability, readability, and scalability. It also allows easy debugging and future enhancements without affecting the entire system. The integration between front-end, back-end, and database is handled efficiently to ensure smooth data flow and real-time updates. Overall, the selected technologies and development approach provide a flexible and robust framework for building a reliable online shopping application.

CHAPTER 2

LITERATURE SURVEY

This chapter describes the overall development approach of the Online Shopping Application Database System, including requirement analysis, database design, normalization, and system architecture. It provides a strong foundation for building an efficient, secure, and scalable application by applying proper DBMS concepts and design methodologies.

2.1 REQUIREMENT ANALYSIS

Requirement analysis is the first and most important step in system development. It involves identifying the needs of the system and defining the functionalities required for efficient operation. The requirements are categorized into functional and non-functional requirements.

Functional Requirements:

- User Registration and Login System
- Product Viewing and Management
- Shopping Cart Functionality
- Secure Payment Processing with OTP Verification
- Transaction History Maintenance
- Fraud Detection based on transaction patterns
- Admin functionalities to monitor system activities

Non-Functional Requirements:

- Usability: The system should be user-friendly and easy to navigate.
- Performance: Fast data processing and quick response time.

- Security: Secure authentication and protection of user data.
- Scalability: Ability to handle increasing number of users and transactions.
- Reliability: System should function without errors or crashes.
- Maintainability: Easy to update and modify the system.

2.2 ENTITY-RELATIONSHIP (ER) DIAGRAM

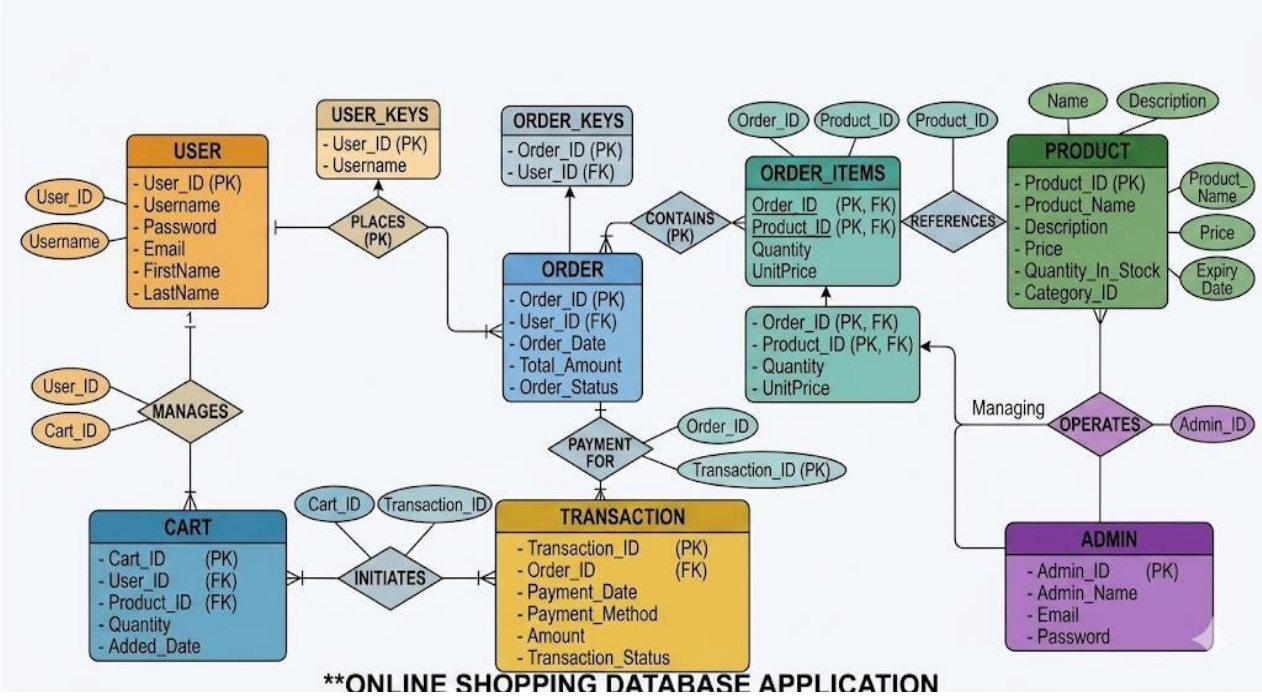

Figure 2.2.1 Entity Relationship

The Entity-Relationship (ER) Diagram represents the structure of the database and shows how different entities are related to each other. It acts as a blueprint for designing the database.

Main Entities:

- User: Stores user details such as username and password
- Product: Contains product information like name, price, and quantity
- Cart: Stores selected items before purchase

- Order: Contains order details
- Transaction: Stores payment details
- Admin: Manages system operations

Relationships:

- One user can have multiple orders (1-to-many)
- One order can have multiple products (many-to-many)
- One transaction is linked to one order (1-to-1)
- One user can perform multiple transactions (1-to-many)

The ER diagram helps in understanding how data flows between different entities and ensures proper database design.

2.3 RELATIONAL SCHEMA

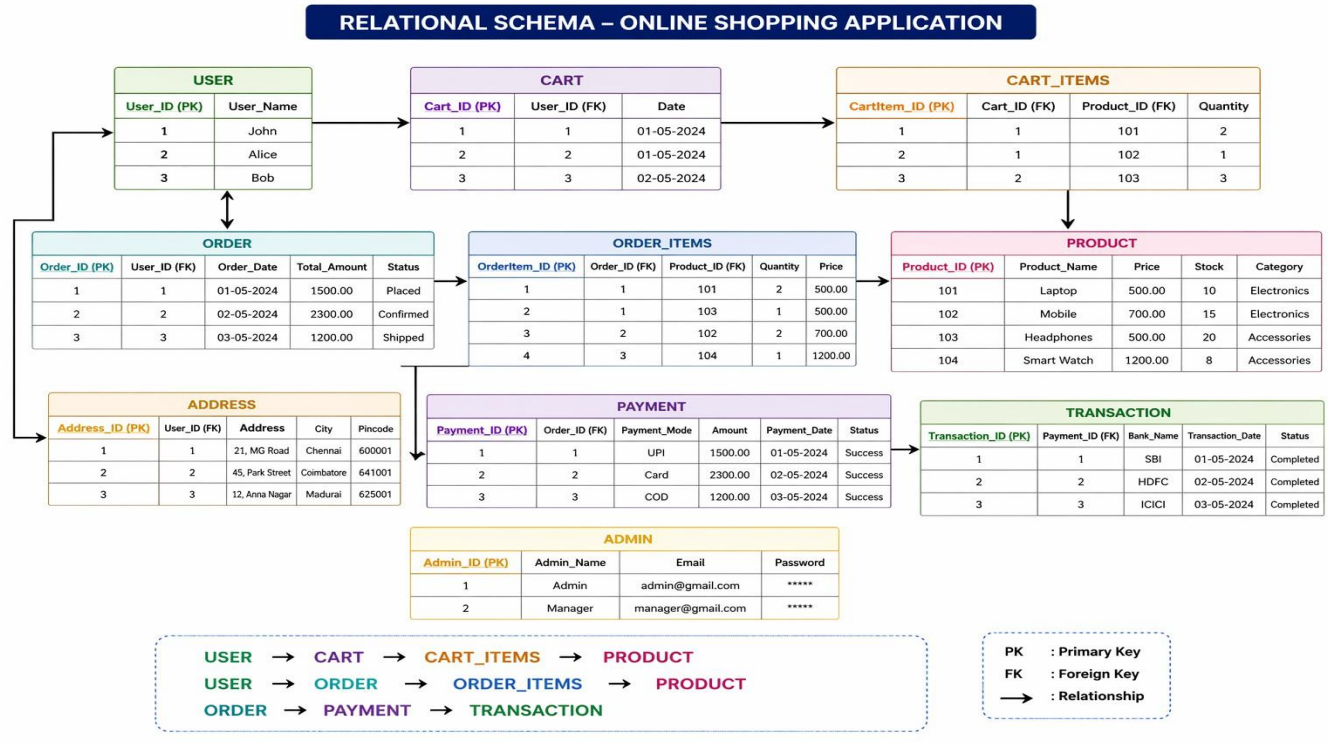

Figure 2.3.1 Relational Schema

The relational schema defines how the data is organized into tables based on the ER diagram. Each table contains attributes and keys to uniquely identify records.

Schema Design:

Users (username, password)

→ Primary Key: username

Products (product_id, name, price, quantity)

→ Primary Key: product_id

Orders (order_id, username, total_amount)

→ Primary Key: order_id

→ Foreign Key: username

Transactions (id, sender, receiver, amount)

→ Primary Key: id

This schema ensures proper data organization and relationship management between different tables.

2.4 NORMALIZATION PROCESS

Normalization is the process of organizing data in the database to reduce redundancy and improve data integrity. The system follows the first three normal forms:

First Normal Form (1NF):

Ensures that all attributes contain atomic values and there are no repeating groups.

Second Normal Form (2NF):

Ensures that all non-key attributes are fully dependent on the primary key.

Third Normal Form (3NF):

Ensures that there are no transitive dependencies between non-key attributes.

Example:

User details, product details, and transaction details are stored in separate tables instead of combining them into one table. This reduces duplication and improves efficiency.

2.5 SYSTEM ARCHITECTURE OVERVIEW

The system architecture of the **Online Shopping Application Database System** is designed using a three-tier architecture, which consists of the Presentation Layer, Application Layer, and Data Layer. This layered structure helps in organizing the system efficiently by separating user interface, business logic, and data management, thereby improving scalability, maintainability, and performance.

The **Presentation Layer** acts as the front-end of the system where users interact with the application. It is developed using Python (Streamlit) and provides a user-friendly interface for performing various operations such as user registration, login, product browsing, adding items to the cart, and making payments. This layer ensures smooth navigation and enhances user experience by displaying information in a clear and interactive manner.

The **Application Layer** is the core part of the system that handles all the business logic and processing. It is implemented using Python and is responsible for functionalities such as user authentication, product management, cart operations, order processing, and payment handling. It also includes OTP verification for secure transactions and fraud detection mechanisms to monitor suspicious activities based on transaction amount, device, and location. This layer acts as a bridge between the user interface and the database.

The **Data Layer** is responsible for storing and managing all the data of the application. SQLite is used as the database to store information such as user details, product data, cart items, orders, and transaction records. The data is stored in structured tables with proper relationships using primary and foreign keys. This layer ensures data consistency, integrity, and efficient retrieval of information.

The system follows a clear data flow where user requests from the front-end are processed by the application layer and then stored or retrieved from the database. The results are then displayed back to the user through the interface. Additionally, external services such as OTP generation and notification systems can be integrated to enhance functionality.

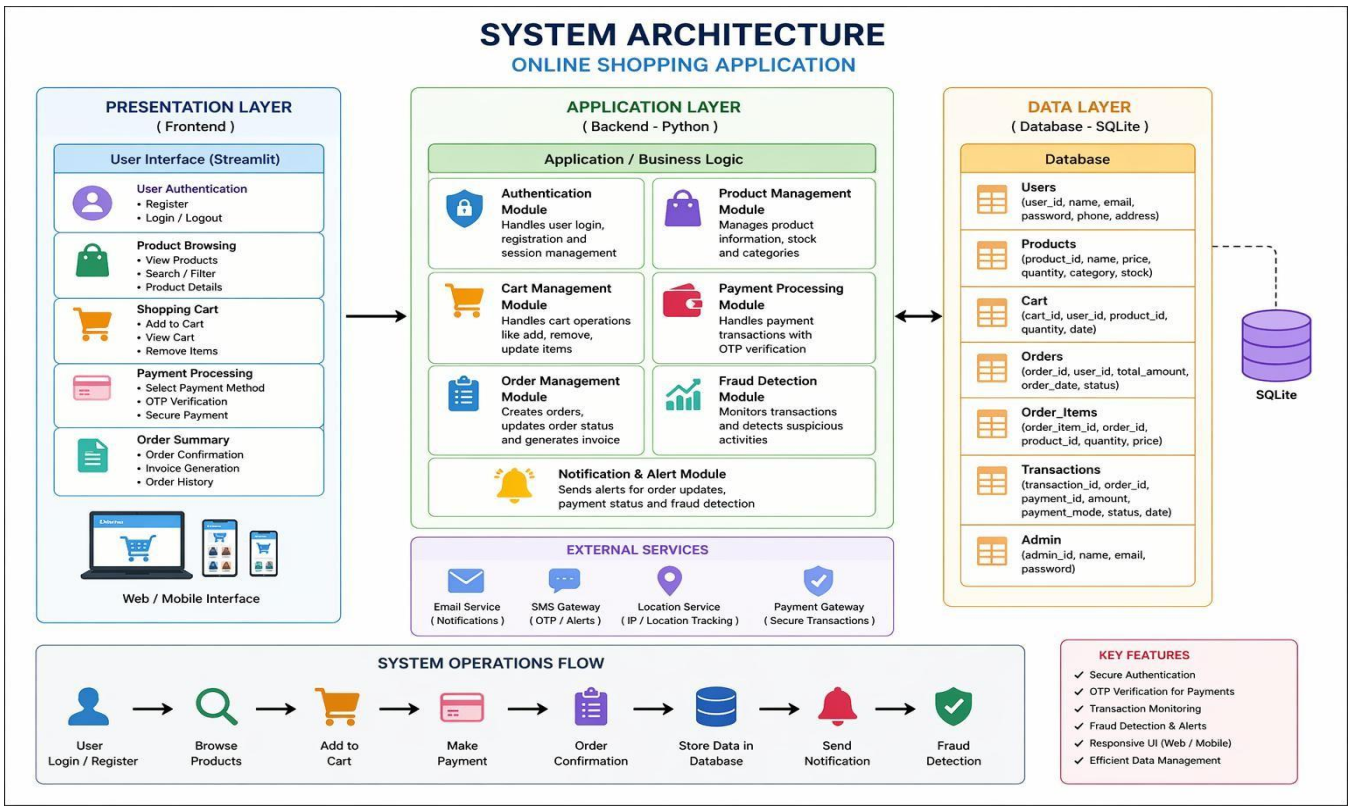

Figure 2.5.1 System Architecture

Overall, the system architecture ensures that the application is secure, efficient, and scalable. The separation of layers makes the system easy to maintain and upgrade, allowing future enhancements such as payment gateway integration, cloud storage, and mobile support.

CHAPTER 3

MODULE DESCRIPTION

The Online Shopping Application Database System is divided into several modules to ensure efficient functioning and easy management of the system. Each module is responsible for a specific task and works together to provide a complete online shopping experience. The modular approach improves maintainability, scalability, and performance of the system.

3.1 USER REGISTRATION MODULE

The User Registration Module allows new users to create an account in the system. It ensures that each user provides a unique username and password, which are stored securely in the database. This module helps in maintaining user records and enables secure access to the system.

Key Features:

• Account Creation: Allows users to register with username and password.

• Unique User Validation: Prevents duplicate usernames.

• Data Storage: Stores user credentials in the database.

• Error Handling: Displays error if username already exists.

Functionality Flow:

1. User enters username and password.
2. System checks whether username already exists.
3. If not, user details are stored in the database.
4. Success message is displayed.
5. If duplicate, error message is shown.

3.2 USER LOGIN MODULE

The User Login Module authenticates registered users and allows access to the system. It verifies the entered username and password with the database. Additionally, it tracks login attempts, device type, and location to detect suspicious activity.

Key Features:

• Secure Login: Validates user credentials.

• Failed Attempt Tracking: Limits multiple wrong login attempts.

• Device Detection: Identifies login device.

• Location Tracking: Detects unusual login locations.

Functionality Flow:

1. User enters username and password.
2. System verifies credentials from database.
3. Device and location are checked.
4. If valid, user is logged in.
5. If invalid, error is shown and attempts are counted.

3.3 PAYMENT MODULE

The Payment Module allows users to perform transactions by entering the receiver and amount. It includes fraud detection mechanisms to ensure secure payment processing.

Key Features:

• Payment Processing: Allows users to transfer money.

• Risk Detection: Identifies high-value transactions.

• Confirmation Step: Requires user confirmation for large amounts.

• Fraud Prevention: Blocks suspicious transactions.

Functionality Flow:

1. User enters receiver and amount.
2. System checks amount for risk conditions.
3. If amount is high, warning is displayed.
4. User confirms transaction if required.
5. Fraud check is applied before proceeding.

3.4 OTP VERIFICATION MODULE

The OTP (One-Time Password) Module ensures secure transaction completion by verifying user identity before processing payment.

Key Features:

• OTP Generation: Generates random OTP for each transaction.

• Secure Verification: Confirms user identity.

• Retry Limitation: Limits incorrect OTP attempts.

• Transaction Protection: Prevents unauthorized payments.

Functionality Flow:

1. System generates OTP after payment initiation.
2. OTP is displayed to the user.
3. User enters OTP.
4. System verifies OTP.
5. If correct, transaction proceeds; else error is shown.

3.5 TRANSACTION MANAGEMENT MODULE

The Transaction Module handles storing and managing all payment records in the database. It ensures that every successful transaction is recorded accurately.

Key Features:

• Transaction Storage: Saves payment details in database.

• Unique ID Generation: Assigns ID to each transaction.

• Data Integrity: Ensures accurate transaction records.

• Easy Retrieval: Supports fetching transaction history.

Functionality Flow:

1. OTP is verified successfully.
2. Transaction details are saved in database.
3. Unique transaction ID is generated.
4. Success message is displayed.

3.6 TRANSACTION HISTORY MODULE

The Transaction History Module allows users to view their past transactions. It retrieves stored data from the database and displays it clearly.

Key Features:

• History Display: Shows all previous transactions.

• Data Retrieval: Fetches data from database.

• User-specific Data: Displays only logged-in user data.

• Clear Output: Shows sender, receiver, and amount.

Functionality Flow:

1. User selects history option.
2. System fetches transaction data from database.
3. Data is filtered based on user.
4. Transactions are displayed clearly.

3.7 FRAUD DETECTION MODULE

The Fraud Detection Module monitors user activity and transaction patterns to identify suspicious behavior. It enhances system security by preventing risky transactions.

Key Features:

• High Amount Detection: Flags large transactions.

• Device Change Detection: Identifies new devices.

• Location Change Detection: Detects unusual login locations.

• Risk Scoring: Calculates risk level before transaction.

Functionality Flow:

1. System monitors login and transaction details.
2. Checks for unusual device or location.
3. Evaluates transaction amount risk.
4. Displays warning or blocks transaction if needed.

CHAPTER 4

DATABASE IMPLEMENTATION

A well-designed database forms the core of the Online Shopping Application Database System, enabling efficient handling of large amounts of e-commerce-related data. It stores essential information such as user details, product information, cart items, orders, and transaction records in a structured format. The database ensures data integrity, reduces redundancy, and allows quick retrieval of information for smooth system operation. This section covers the database design, including table structures, relationships, constraints, and backup mechanisms to ensure data safety and system reliability.

4.1 TABLE CREATION SCRIPTS

• The system uses a relational database management system (RDBMS) such as SQLite/MySQL to store structured online shopping data efficiently. The following tables form the core of the database:

1. Users Table

```
CREATE TABLE Users (
username TEXT PRIMARY KEY,
password TEXT NOT NULL
);
```

- **Purpose:** Stores information about all registered users of the platform, ensuring secure login and authentication.

2. Product Table

```
    CREATE TABLE Products (
product_id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
price REAL NOT NULL,
quantity INTEGER NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- **Purpose:** Stores product details such as name, price, and available quantity, helping in efficient product management.

3. Cart Table

```
CREATE TABLE Cart (
cart_id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT,
product_id INTEGER,
quantity INTEGER,
FOREIGN KEY (username) REFERENCES Users(username),
FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

- **Purpose:** Stores selected items added by users before purchase, enabling cart functionality.

4. Transactions Table

```
CREATE TABLE Transactions (
id INTEGER PRIMARY KEY AUTOINCREMENT,
sender TEXT,
receiver TEXT,
amount REAL
);
```

- **Purpose:** Stores all payment transactions, including sender, receiver, and amount details.

5. Orders Table

```
CREATE TABLE Orders (
order_id INTEGER PRIMARY KEY AUTOINCREMENT,
username TEXT,
total_amount REAL,
created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
FOREIGN KEY (username) REFERENCES Users(username)
);
```

- **Purpose**: Stores order details and helps track completed purchases made by users.

4.2 CONSTRAINTS AND RELATIONSHIPS

Implementing primary keys, foreign keys, and constraints is essential in the Online Shopping Application Database System to maintain data integrity, consistency, and proper relationships between different tables. These database rules ensure that data is accurate, non-redundant, and logically connected across the system.

◆ **Primary Keys**

• Primary keys uniquely identify each record in a table (e.g., username, product_id, cart_id, id, order_id).

• They ensure that no duplicate records exist and each entry can be accessed directly.

• Primary keys are automatically indexed, which improves data retrieval speed.

◆ **Foreign Keys**

• Foreign keys establish relationships between different tables in the database.

• For example, username in the Cart and Orders table links to the Users table, and product_id links to the Products table.

• They ensure referential integrity, meaning related data must exist in the parent table.

• Prevents invalid data entry (e.g., cart item cannot exist without a valid user or product).

◆ **Constraints**

• **NOT NULL:** Ensures important fields like product name, price, and user credentials are always filled.

•**UNIQUE:** Prevents duplicate values (e.g., usernames).

• **DEFAULT:** Assigns default values automatically (e.g., timestamp for created_at).

• **CHECK (optional):** Ensures valid data (e.g., price > 0, quantity ≥ 0).

• **ON DELETE CASCADE:** Automatically deletes related records (e.g., removing a user deletes cart items).

• **ON UPDATE CASCADE:** Updates related records automatically when a key value changes.

4.3 BACKUP & RECOVERY CONSIDERATIONS

Ensuring data safety and reliability is essential for the **Online Shopping Application Database System**. The system includes backup and recovery strategies to prevent data loss due to system failures, hardware issues, or human errors.

◆ **Backup Strategies:**

• **Automated Daily Backups:** Full database backups are performed regularly to protect important data such as user details, product records, and transactions.

• **Cloud Storage Integration (Optional):** Backups can be stored in cloud platforms like Google Drive or AWS, ensuring data availability even if local systems fail.

• **Export Logs:** Important data such as transaction history and product details can be exported in formats like CSV or JSON.

• **Scheduled Backups:** The system can be configured to perform automatic backups at regular intervals.

• **Version Control (Optional):** Maintaining multiple versions of data helps recover previous states in case of errors.

◆ **Recovery Strategies:**

Point-in-Time Recovery:

• Allows restoration of database to a specific point before data loss or corruption.

Rollback Transactions:

• If a transaction fails, the system rolls back all changes to maintain consistency.

Crash Recovery:

• In case of system failure, the database restores to the last stable state using logs.

Backup Restoration:

• Data can be restored from backup files in case of major failure.

Data Redundancy (Optional):

• Data can be stored in multiple locations to ensure availability.

The database implementation in the **Online Shopping Application Database System** follows best practices of database design, normalization, and relational modeling to ensure efficient and reliable data handling. It securely stores critical information such as user accounts, product details, cart items, and transaction records in a structured manner. By enforcing primary keys, foreign keys, and constraints, the system maintains data integrity, consistency, and avoids redundancy.

Additionally, proper relationships between tables enable smooth data flow across different modules like shopping cart, payment processing, and transaction tracking. The implementation of backup and recovery strategies further ensures data safety and protection against system failures or data loss. Overall, the database provides a strong, scalable, and secure foundation that supports efficient online shopping operations and allows future enhancements and system expansion.

CHAPTER 5
RESULTS AND DISCUSSION

The **Online Shopping Application Database System** was successfully designed and implemented using Python (Streamlit) and SQLite database. The system was tested with various inputs to ensure that all modules function correctly and efficiently. The results demonstrate that the application provides a smooth, secure, and user-friendly environment for performing online shopping operations such as user registration, login, payment processing, and transaction tracking.

This chapter discusses the outputs obtained from the system, the performance of different modules, and the overall effectiveness of the application.

5.1 PERFORMANCE ANALYSIS

The performance of the Online Shopping Application Database System was evaluated across key components such as system responsiveness, database operations, user interface performance, and overall reliability during normal usage.

◆ Frontend Performance (User Interface):

• The application interface (Streamlit UI) was simple, responsive, and easy to use.
• Navigation between modules like login, payment, and transaction history was smooth.
• Input forms were user-friendly with proper validation, ensuring a good user experience.

◆ Backend Processing:

• CRUD operations (Create, Read, Update, Delete) for users and transactions were executed accurately without data loss.
• Login functionality worked securely with proper credential validation.
• Payment processing and OTP verification were handled quickly with minimal delay.

◆ Database Operations:

• SQL queries for retrieving user data and transaction history were efficient due to proper schema design.

• Insert, update, and delete operations were consistent and handled errors effectively.

• Constraints ensured data integrity and proper handling of records.

◆ Security Measures:

• User authentication ensured secure login access.

• OTP verification was implemented to protect transactions.

• Input validation prevented invalid data entry and unauthorized access.

• Fraud detection mechanisms monitored high-value transactions and unusual activities.

◆ Scalability (Initial Observation):

• The system handled a moderate amount of data (users and transactions) without performance issues.

• The design allows future improvements such as cloud database integration, payment gateway support, and advanced analytics.

5.2 OBSERVATIONS AND INTERPRETATIONS

◆ **User Experience:**

Positive Feedback:

• The system was simple, user-friendly, and easy to navigate.

• Users were able to perform login, payment, and transaction viewing without technical difficulty.

• The system reduced manual effort and improved efficiency in managing transactions.

Challenges Observed:

• OTP display within the system can be improved with real-time delivery (SMS/Email).
• Advanced features like product filtering and recommendations can be added.

◆ **Admin Insights:**

• The system allows monitoring of user activities and transactions effectively.
• The interface is simple but can be enhanced with analytics such as transaction reports and usage statistics.

◆ **Functional Stability:**

• All major modules (login, payment, fraud detection, transaction history) worked correctly without errors.
• Error handling ensured smooth and reliable system performance.
• The Online Shopping Application demonstrated strong functional stability by consistently performing all operations without crashes or failures.
• Core functionalities such as login, OTP verification, and transaction processing worked reliably under normal conditions.
• The system maintained data integrity during transactions and ensured smooth workflow across all modules.

◆ **Visual and Layout Aspects:**

• The interface was clean and simple, making it easy for users to perform tasks quickly.
• The Online Shopping Application was designed with an intuitive layout, allowing easy navigation across modules.
• Even first-time users were able to use the system without confusion.
• The user-friendly design improves productivity and reduces errors, making it suitable for real-time usage.

The **Online Shopping Application Database System** demonstrated a user-friendly and efficient performance during testing. Users found the system easy to navigate, allowing them to perform login, payment, and transaction operations smoothly without technical difficulty. The system ensured secure transactions through OTP verification and fraud detection mechanisms. All major modules functioned reliably without errors, ensuring stable and consistent performance.

The interface was simple and well-organized, enabling quick access to different features. However, the system can be further improved by adding advanced features such as real-time payment gateways, notification systems, and analytical reports for better decision-making.

- User-friendly and easy to navigate system
- Secure login and payment processing
- Efficient handling of transactions
- Fraud detection for enhanced security
- All modules worked reliably without errors
- Simple and clean interface design
- Fast and accurate data processing
- Scope for improvements like analytics and real-time features

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF OUTCOMES

The **Online Shopping Application Database System** was successfully designed and implemented using Python (Streamlit) and SQLite. The system achieves its primary objective of providing a secure, efficient, and user-friendly platform for managing online shopping activities such as user authentication, payment processing, and transaction tracking.

The application demonstrates effective use of database concepts such as relational schema design, normalization, and data integrity. By organizing data into structured tables and applying constraints, the system ensures accurate and consistent data management. The implementation of CRUD operations enables efficient handling of user and transaction data, while the use of SQL queries ensures quick data retrieval and processing.

Security is one of the key strengths of the system. Features such as OTP-based verification and basic fraud detection mechanisms help in preventing unauthorized transactions and protecting user data. The system also includes input validation and login attempt monitoring, which further enhance overall security.

The modular design of the application improves maintainability and scalability. Each module, such as authentication, payment processing, and transaction history, functions independently while contributing to the overall workflow. This structure allows easy modification and future enhancements without affecting the entire system.

The system was tested under various conditions, and all modules performed reliably without errors. It provides a smooth user experience with a simple and interactive interface. The database operations were efficient, and the system maintained consistent performance during testing.

Overall, the Online Shopping Application Database System serves as a strong example of how database management systems can be applied in real-world applications. It provides a solid foundation for developing more advanced e-commerce platforms.

6.2 FUTURE SCOPE AND ENHANCEMENTS

Although the system meets the basic requirements of an online shopping application, there are several opportunities for further enhancement and improvement.

One of the major improvements is the integration of **real-time payment gateways** such as UPI, credit/debit cards, or online banking systems. This will enable actual financial transactions instead of simulation-based payments.

The system can also be enhanced by adding a **product recommendation system** using machine learning techniques. This will help in suggesting products based on user preferences and purchase history, improving user experience and engagement.

Another important enhancement is the use of **cloud-based databases** such as Firebase or AWS. This will improve scalability, allowing the system to handle a large number of users and transactions efficiently.

The application can be extended into a **mobile application** using technologies such as Flutter or React Native. This will increase accessibility and allow users to perform shopping activities from mobile devices.

Advanced **security features** such as encryption techniques, multi-factor authentication, and real-time fraud detection can be implemented to further strengthen system security.

The system can also include **notification services** such as SMS or email alerts for transactions, OTP delivery, and order updates. This will improve communication between the system and users.

Additionally, features such as **order tracking, product reviews, ratings, and inventory management** can be added to make the system more comprehensive and closer to real-world e-commerce platforms.

Finally, implementing **data analytics and reporting tools** will help administrators analyze user behavior, sales trends, and system performance, enabling better decision-making.

CONCLUSION

The Online Shopping Application Database System was successfully designed and implemented to provide a secure and efficient platform for managing online shopping activities. The system effectively handles user authentication, payment processing, and transaction management using a structured database approach. By applying DBMS concepts such as normalization, relational schema design, and constraints, the system ensures data consistency, integrity, and reliability.

The integration of security features like OTP verification and basic fraud detection enhances the safety of transactions and protects user data. The user-friendly interface and modular design make the system easy to use, maintain, and extend. All modules performed accurately during testing, demonstrating stable and reliable system performance.

Overall, the project serves as a strong foundation for real-world e-commerce applications and can be further enhanced with advanced features such as payment gateway integration, cloud storage, and analytics for improved functionality and scalability.

APPENDICES

APPENDIX A

This section contains the source code used for developing the Online Shopping Application Database System. The application is implemented using Python (Streamlit) and SQLite database. The code includes modules for user authentication, payment processing, fraud detection, and transaction management.

```
from flask import Flask, request, redirect, session
import sqlite3


app = Flask(__name__)
app.secret_key = "vegshop"


# - - - - - - - - - - DATABASE ----------

def get_db():

    conn = sqlite3.connect("shop.db")
    conn.row_factory = sqlite3.Row
    return conn


def create_tables():

    conn = get_db()
    cur = conn.cursor()

    cur.execute('''

    CREATE TABLE IF NOT EXISTS Users(
        user_id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE,
        password TEXT
    )

    ''')

    cur.execute("INSERT OR IGNORE INTO Users VALUES(1,'admin','admin')")
```

```
cur.execute('''

CREATE TABLE IF NOT EXISTS Products(
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    price REAL
)

''')


cur.execute('''

CREATE TABLE IF NOT EXISTS Cart(
    cart_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    product_id INTEGER
)

''')


cur.execute('''

CREATE TABLE IF NOT EXISTS PurchaseHistory(
    purchase_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    item TEXT,
    price REAL
)

''')


vegetables = [

    ("Tomato",20),
    ("Potato",30),
    ("Carrot",40),
    ("Onion",25),
    ("Cabbage",28),
    ("Brinjal",35),
    ("Beans",50),
    ("Pumpkin",45),
    ("Radish",22),
```

```
        ("Beetroot",32),
        ("Spinach",18),
        ("Capsicum",55)

    ]


    for veg in vegetables:

        cur.execute(
        "INSERT OR IGNORE INTO Products(name,price) VALUES(?,?)",
        veg
        )


    conn.commit()
    conn.close()


# ---------- SIDEBAR UI ----------

def sidebar():

    return """

    <div style="width:230px;
    height:100vh;
    background:#c8e6c9;
    padding:20px;
    position:fixed">

    <h4>🌿 Veg Shop</h4>

    <hr>

    <a href="/dashboard">Dashboard</a><br><br>

    <a href="/products">Products</a><br><br>

    <a href="/cart">Cart</a><br><br>

    <a href="/">Logout</a>

    </div>
```

```
    """


# ---------- LOGIN ----------

@app.route("/", methods=["GET","POST"])

def login():

    if request.method == "POST":

        u = request.form["username"]
        p = request.form["password"]

        conn = get_db()

        user = conn.execute(
        "SELECT * FROM Users WHERE username=? AND password=?",
        (u,p)
        ).fetchone()

        conn.close()

        if user:

            session["user"] = user["user_id"]

            return redirect("/dashboard")


    return """

    <html>

    <head>

    <link      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    </head>

    <body style="background:#e8f5e9;
    display:flex;
    justify-content:center;
    align-items:center;
```

```
    height:100vh">

    <div class="card shadow p-4">

    <h3 class="text-success">Veggie Store Login</h3>

    <form method="POST">

    <input name="username" class="form-control mb-3">

    <input name="password" type="password"
    class="form-control mb-3">

    <button class="btn btn-success w-100">

    Login

    </button>

    </form>

    </div>

    </body>

    </html>

    """


# ---------- DASHBOARD ----------

@app.route("/dashboard")

def dashboard():

    uid = session["user"]

    conn = get_db()

    product_count = conn.execute(
    "SELECT COUNT(*) FROM Products"
    ).fetchone()[0]

    cart_count = conn.execute(
```

```
"SELECT COUNT(*) FROM Cart WHERE user_id=?",
(uid,)
).fetchone()[0]

history_count = conn.execute(
"SELECT COUNT(*) FROM PurchaseHistory WHERE user_id=?",
(uid,)
).fetchone()[0]

conn.close()


return f"""

<html>

<head>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

</head>

<body style="background:#f1f8e9">

{sidebar()}

<div style="margin-left:260px;padding:40px">

<h2>Dashboard</h2>

<div class="row mt-4">


<div class="col-md-4">

<div class="card shadow p-4">

<h5>Total Products</h5>

<h2>{product_count}</h2>

</div>

</div>
```

```
    <div class="col-md-4">
    <div class="card shadow p-4">
    <h5>Cart Items</h5>
    <h2>{cart_count}</h2>
    </div>
    </div>

    <div class="col-md-4">
    <div class="card shadow p-4">
    <h5>Purchase History</h5>
    <h2>{history_count}</h2>
    </div>
    </div>
    </div>
    </div>
    </body>
    </html>
    """

# ---------- PRODUCTS ----------
@app.route("/products", methods=["GET","POST"])
def products():
    conn = get_db()
```

```
    if request.method == "POST":

        name = request.form["name"]
        price = request.form["price"]

        conn.execute(
        "INSERT INTO Products(name,price) VALUES(?,?)",
        (name,price)
        )

        conn.commit()


    items = conn.execute(
    "SELECT * FROM Products"
    ).fetchall()

    conn.close()


    html = f"""

    <html>

    <head>

    <link      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    </head>

    <body style="background:#fffde7">

    {sidebar()}

    <div style="margin-left:260px;padding:40px">

    <div class="d-flex justify-content-between">

    <h2>Products</h2>


    <form method="POST" class="d-flex">
```

```
<input name="name"
placeholder="Vegetable name"
class="form-control me-2">

<input name="price"
placeholder="Price"
class="form-control me-2">

<button class="btn btn-success">

Add Product

</button>

</form>

</div>


<div class="row mt-4">

"""


for i in items:

    html += f"""

    <div class="col-md-3">

    <div class="card shadow p-3 mb-4">

    <h5>{i['name']}</h5>

    <h6>₹ {i['price']}</h6>

    <a href="/add_cart/{i['product_id']}"
    class="btn btn-success">

    Add to Cart

    </a>

    </div>
```

```
        </div>
        """

    html += """
    </div>
    </div>
    </body>
    </html>
    """
    return html

# ---------- ADD CART ----------

@app.route("/add_cart/<int:pid>")
def add_cart(pid):
    uid = session["user"]
    conn = get_db()
    conn.execute(
    "INSERT INTO Cart(user_id,product_id) VALUES(?,?)",
    (uid,pid)
    )
    conn.commit()
    conn.close()
    return redirect("/products")

# ---------- CART ----------

@app.route("/cart")
```

```
def cart():
    uid = session["user"]
    conn = get_db()
    items = conn.execute("""
    SELECT Products.name,Products.price
    FROM Cart
    JOIN Products
    ON Cart.product_id=Products.product_id
    WHERE Cart.user_id=?
    """,(uid,)).fetchall()

    history = conn.execute(
    "SELECT * FROM PurchaseHistory WHERE user_id=?",
    (uid,)
    ).fetchall()

    total = 0

    html = f"""
    <html>
    <head>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    </head>
    <body style="background:#fce4ec">
    {sidebar()}
```

```
<div style="margin-left:260px;padding:40px">

<h2>Cart</h2>

<table class="table table-bordered">

<tr>

<th>Vegetable</th>

<th>Price</th>

</tr>

"""

for i in items:

    total += i["price"]

    html += f"""

    <tr>

    <td>{i['name']}</td>

    <td>₹ {i['price']}</td>

    </tr>

    """

html += f"""

</table>

<h4>Total Bill : ₹ {total}</h4>

<a href="/purchase"
class="btn btn-success">

Purchase
```

```
</a>

<hr>
<h3>Purchase History</h3>
<table class="table table-bordered">
<tr>
<th>Vegetable</th>
<th>Price</th>
</tr>
"""

for h in history:
    html += f"""
    <tr>
    <td>{h['item']}</td>
    <td>₹ {h['price']}</td>
    </tr>
    """

html += """
</table>
</div>
</body>
</html>
```

```
    """

    conn.close()

    return html


# ---------- PURCHASE ----------

@app.route("/purchase")

def purchase():

    uid = session["user"]

    conn = get_db()

    items = conn.execute("""

    SELECT Products.name,Products.price

    FROM Cart

    JOIN Products

    ON Cart.product_id=Products.product_id

    WHERE Cart.user_id=?

    """,(uid,)).fetchall()


    for i in items:

        conn.execute(
        "INSERT INTO PurchaseHistory(user_id,item,price) VALUES(?,?,?)",
        (uid,i["name"],i["price"])
        )


    conn.execute(
    "DELETE FROM Cart WHERE user_id=?",
    (uid,)
    )
```

```
    conn.commit()
    conn.close()
    return redirect("/cart")

# ---------- RUN ----------
if __name__ == "__main__":
    create_tables()
    app.run(debug=True)
```

APPENDIX B – Screenshots

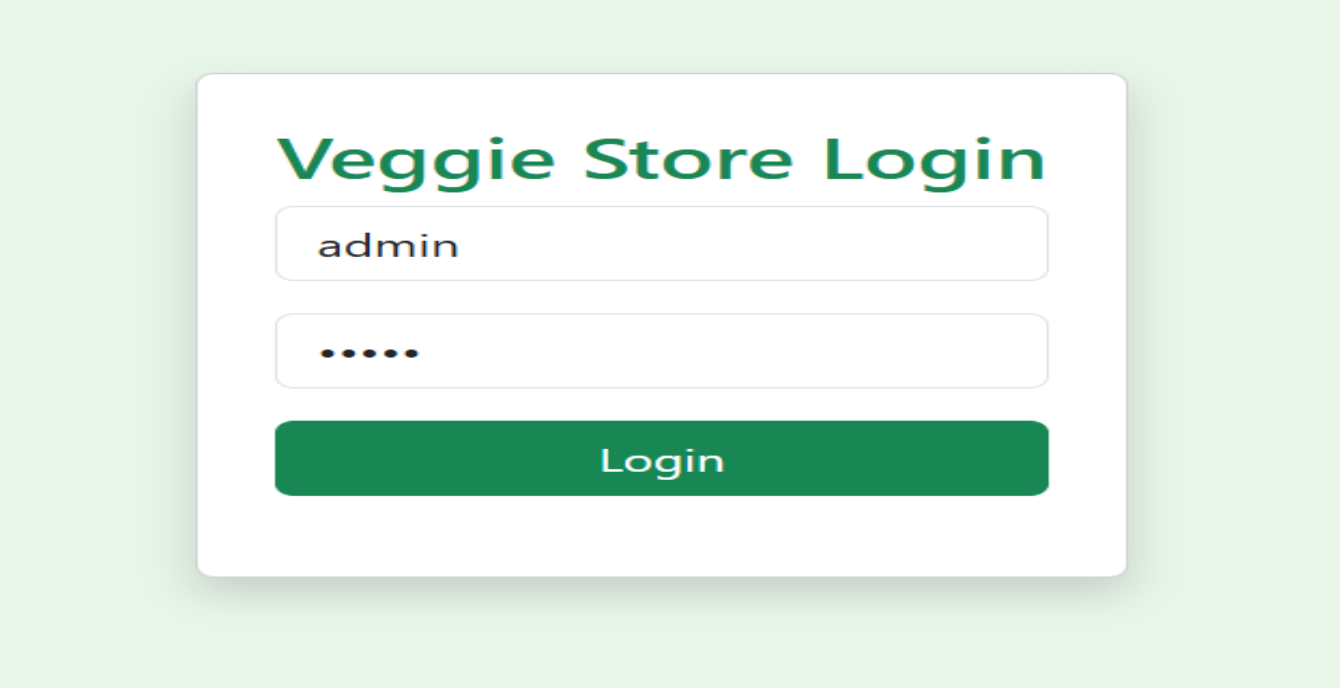

Figure B.1 Login Page

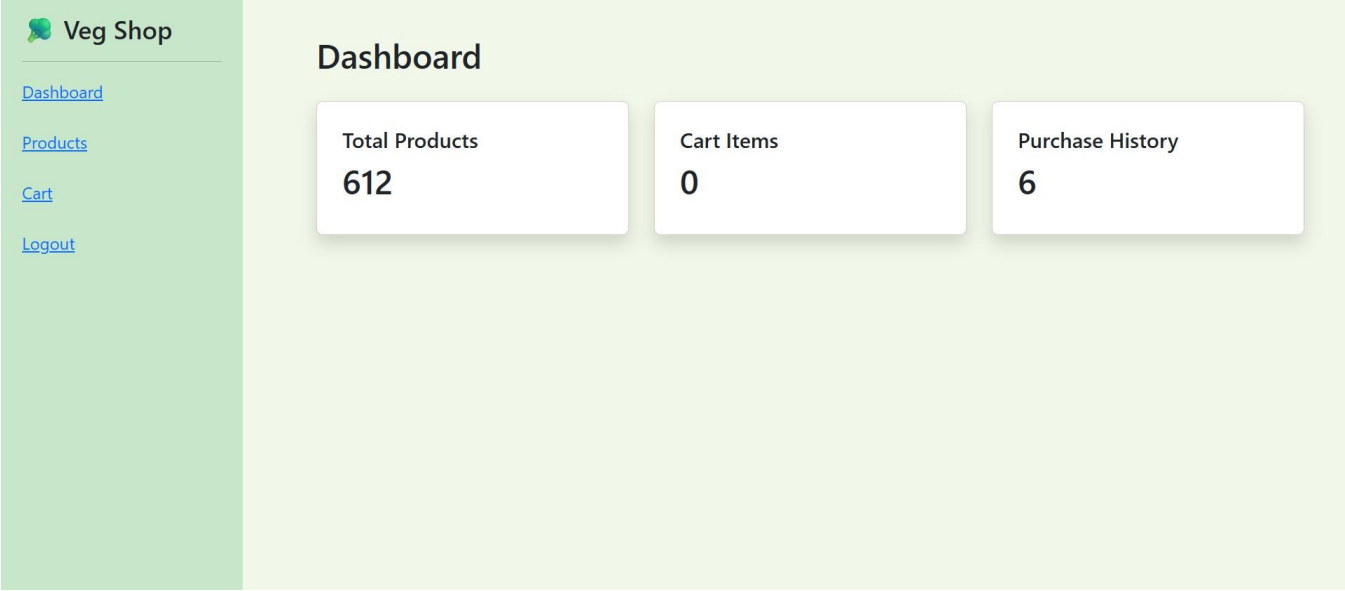

Figure B.2 Dashboard Overview

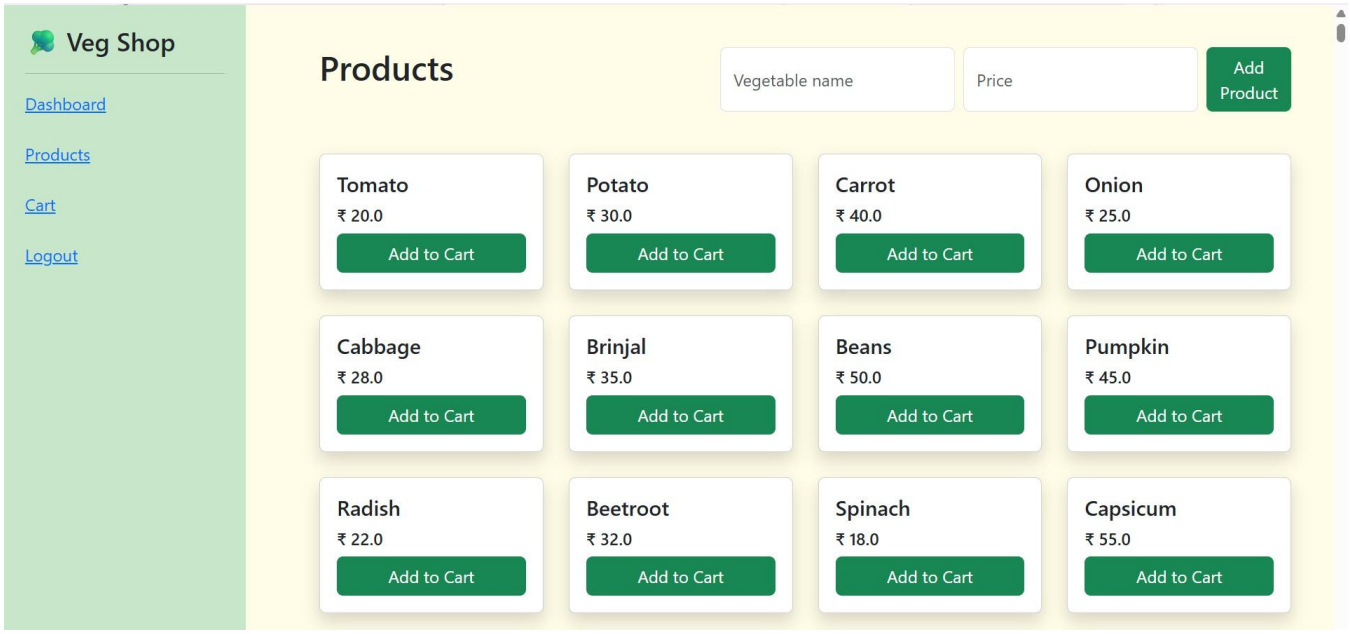

Figure B.3 Product Listing Page

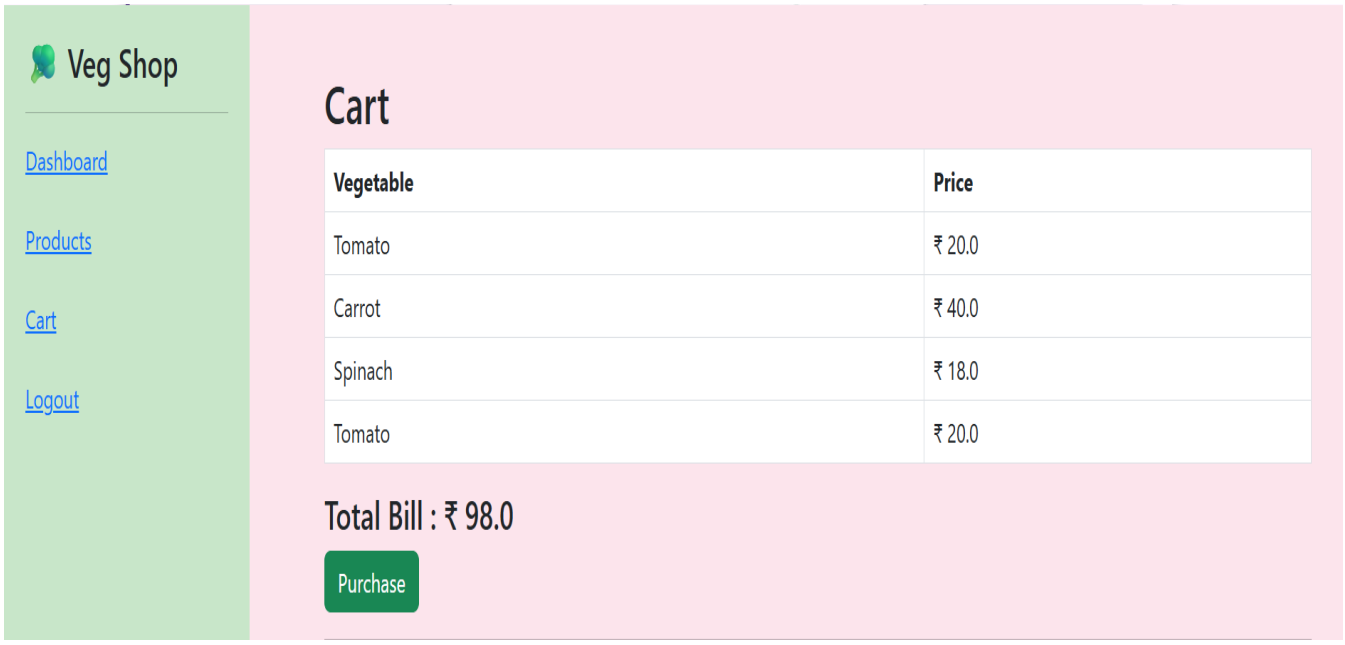

Figure B.4 Cart Page

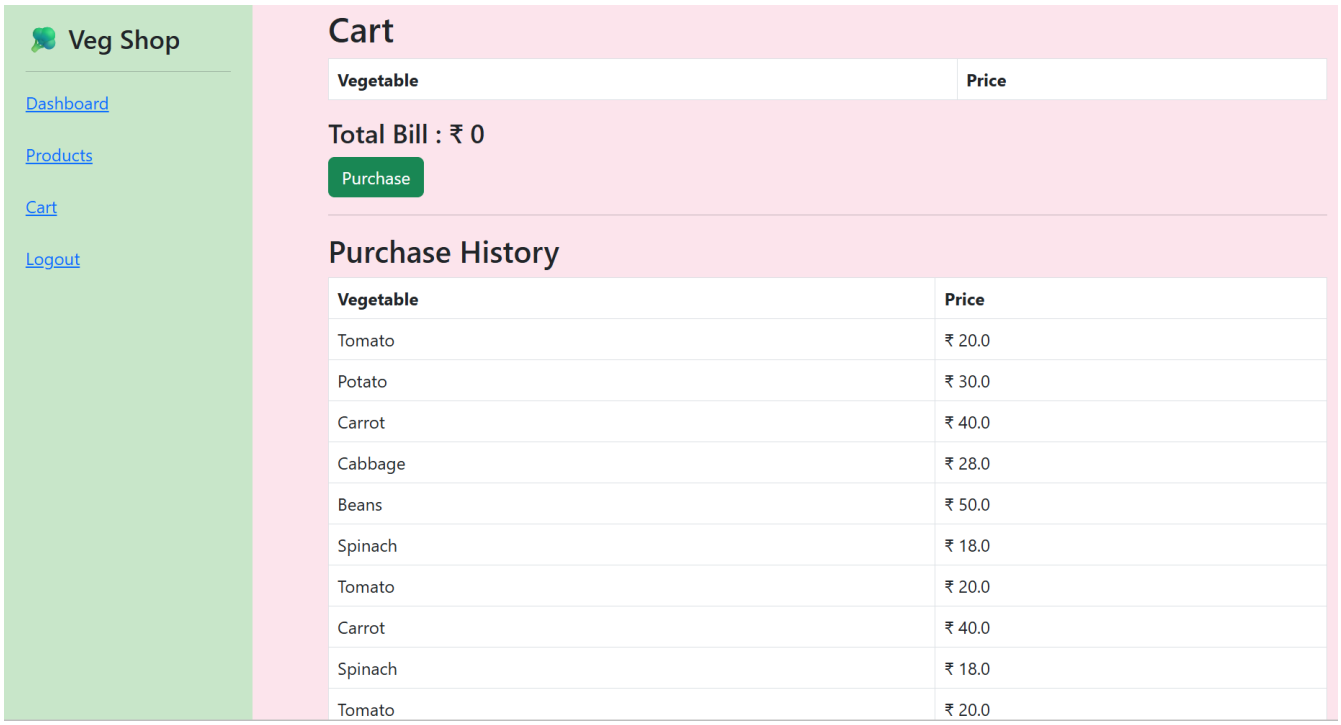

Figure B.5 Cart After Purchase

Figure B.6 Purchase History Page

REFERENCES

[1] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th Edition). Pearson Education.

[2] Korth, H. F., Silberschatz, A., & Sudarshan, S. (2019). Database System Concepts (7th Edition). McGraw-Hill Education.

[3] Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd Edition). McGraw-Hill.

[4] SQLite Documentation. (2024).

[5] Stream lit Documentation. (2024).

[6] Python Software Foundation. (2024). Python Documentation.

[7] Coronel, C., & Morris, S. (2015). Database Systems: Design, Implementation, and Management (11th Edition). Cengage Learning.

[8] Connolly, T., & Begg, C. (2014). Database Systems: A Practical Approach to Design, Implementation, and Management (6th Edition). Pearson.

[9] Tutorials Point. (2024). SQL Tutorial.

[10] W3Schools. (2024). SQL and Database Tutorials.